

操作步骤

训练和保存 KNN 模型的详细步骤

1. 定义数据集路径:

- 代码: `"dataFolder='p_dataset_26';"`

- 解释: 设置数据集存放的文件夹路径。这里的"`p_dataset_26`"是假定的数据集路径。

2. 创建 `imageDatastore` 对象:

- 代码:

`"images=imageDatastore(dataFolder,'IncludeSubfolders',true,'LabelSource','foldernames');"`

- 解释: 创建一个 `imageDatastore` 对象来管理图片数据。这个对象会包含指定文件夹下的所有图片, 并根据文件夹名自动标记图片的标签。

3. 分割数据集为训练集和测试集:

- 代码: `"[trainingSet,testSet]=splitEachLabel(images,0.7,'randomize');"`

- 解释: 将数据集分为 70% 的训练集和 30% 的测试集。"`splitEachLabel`"函数确保每个类别的图像都按照相同的比例分割。

4. 提取特征:

- 代码:

`"trainingFeatures=featureExtractor(trainingSet);"`和`"testFeatures=featureExtractor(testSet);"`

- 解释: 使用"`featureExtractor`"函数从训练集和测试集中提取特征。这些特征将用于训练和测试 KNN 模型。

5. 设置 KNN 分类器参数:

- 代码: `"numNeighbors=5;"`和`"distanceMetric='euclidean';"`

- 解释: 设定 KNN 分类器的参数。这里使用 5 个邻居 ($k=5$) 和欧几里得距离度量。

6. 训练 KNN 分类器:

- 代码:

`"knnClassifier=fitcknn(trainingFeatures,trainingLabels,'NumNeighbors',numNeighbors,'Distance',distanceMetric);"`

- 解释: 用训练集的特征和标签来训练 KNN 分类器。

7. 在测试集上评估分类器:

- 代码: `"predictedLabels=predict(knnClassifier,testFeatures);"`

- 解释: 使用训练好的 KNN 分类器对测试集的特征进行预测, 得到预测标签。

8. 计算并显示准确率:

-代码: `accuracy=sum(predictedLabels==testLabels)/numel(testLabels);`

-解释: 计算测试集上的预测准确率, 并打印出来。

9.保存训练好的模型(可选):

-代码: `save('knnModel.mat','knnClassifier');`

-解释: 将训练好的 KNN 模型保存到".mat"文件中。这步骤在代码中被注释掉了, 如需使用, 需要取消注释。

预测和识别字符的详细步骤

1.加载训练好的 KNN 模型:

-代码: `model=load('knnModel.mat');knnModel=model.knnClassifier;`

-解释: 加载之前保存的 KNN 模型。这里".mat"文件中的变量名称为"knnClassifier"。

2.分割图像并获取边界框:

-代码:

`imagePath='hello_world.png';img=imread(imagePath);`

和 `bboxes=splitAndDisplayBoundingBoxes(imagePath,widthThreshold,areaThreshold,heightThreshold);`

-解释: 读取要识别的图像文件, 并使用"splitAndDisplayBoundingBoxes"函数分割图像, 提取字符的边界框。阈值用于确定字符的大小和形状。

"splitAndDisplayBoundingBoxes"函数详解:

函数"splitAndDisplayBoundingBoxes"的目的是从给定的图像中提取字符的边界框并对这些边界框进行处理。

1.读取图像:

-代码: `img=imread(imagePath);`

-解释: 使用 MATLAB 的"imread"函数读取指定路径的图像。

2.转换为灰度图像:

-代码: `grayImg=rgb2gray(img);`

-解释: 将读取的彩色图像转换为灰度图像。

3.应用阈值化以获得二值图像:

-代码: `binaryImg=imbinarize(grayImg);`

-解释: 对灰度图像应用阈值化操作, 将其转换为二值图像, 以便于后续处理。

4.使用形态学操作以分离字符:

-代码:

`se=strel('rectangle',[1,5]);erodedImg=imerode(binaryImg,se);dilatedImg=imdilate(erodedImg,`

g,se);"

-解释：使用形态学操作（腐蚀和膨胀）来分离图像中的字符。这些操作有助于减少字符间的相连。

5.查找连通组件:

-代码:

```
"[~,L]=bwboundaries(dilatedImg,'noholes');stats=regionprops(L,'BoundingBox','Area');"
```

-解释：识别二值图像中的连通组件，并计算每个组件的边界框和面积。

6.过滤掉小的连通组件:

-代码:

```
"stats=stats([stats.Area]>areaThreshold);stats=stats(arrayfun(@(s)s.BoundingBox(4)>=heightThreshold,stats));stats=stats(arrayfun(@(s)s.BoundingBox(3)>=widthThreshold/2,stats));"
```

-解释：移除面积、高度或宽度小于指定阈值的连通组件，以排除非字符元素。

7.初始化边界框数组:

-代码: "bboxes=[];"

-解释：创建一个空数组用于存储符合条件的边界框。

8.处理每个连通组件的边界框:

-代码：在"for"循环中处理每个连通组件。

-解释：对于每个连通组件，检查其边界框的宽度。如果宽度大于阈值，则将该边界框分割为两个新的边界框（左半部分和右半部分），否则保留原始边界框。

9.将边界框添加到数组中:

-代码: "bboxes=[bboxes;bb1;bb2];"或"bboxes=[bboxes;bb];"

-解释：根据边界框的宽度，将分割后的或原始的边界框添加到"bboxes"数组中。

这个函数最终返回一个包含所有符合条件的边界框的数组。这些边界框随后可以用于字符识别和其他处理。

该函数解释结束

3.初始化识别文本和输出图像:

-代码: "recognizedText='';outputImage=img;"

-解释：初始化一个空字符串来存储识别的文本和一个输出图像。

4.对于每个边界框，提取特征并预测标签:

-代码：包含在"for"循环中的一系列步骤，如裁剪图像、调整大小、保存临时文件、创建"ImageDatastore"、提取特征、使用 KNN 模型预测标签。

-解释：对于每个边界框，首先裁剪出字符图像，调整其大小，并将其保存为临时文件。然后创建"ImageDatastore"对象，使用"featureExtractor"函数提取特征，最后使用 KNN 模型进行预测。

5.将预测标签添加到识别文本和输出图像:

-代码:

"recognizedText=[recognizedText,label];"

和"outputImage=insertText(outputImage,position,label,'AnchorPoint','Center');"

-解释：将预测出的标签添加到识别文本字符串中，并将标签写在输出图像的相应位置上。

6.删除临时文件:

-代码: "delete(tempFileName);"

-解释: 处理完一个字符后，删除其对应的临时文件。

7.输出识别的文本:

-代码: "disp(['识别的文本:',recognizedText]);"

-解释: 在命令窗口显示识别出的完整文本。

8.显示带有文本标签的图像:

-代码: "imshow(outputImage);"

-解释: 显示最终的图像，其中包含了识别出的字符和其在原图中的位置。